AULA PRÁTICA — MYSQL

Sistema de Controle Financeiro Básico

Material para desenvolvimento em sala de aula • Banco de Dados Relacional

Objetivo da aula

Nesta atividade, os alunos irão construir um banco de dados para registrar receitas e despesas. O projeto trabalha conceitos fundamentais de MySQL: **CREATE DATABASE**, **CREATE TABLE**, tipos de dados, chave primária, chave estrangeira, INSERT, SELECT, JOIN, SUM, WHERE, ORDER BY, UPDATE e DELETE.

Modelo do sistema

O sistema possui duas tabelas principais:

Tabela	Responsabilidade
categorias	Armazena os tipos de classificação financeira, como Salário, Alimentação e Transporte.
lancamentos	Armazena cada entrada ou saída de dinheiro realizada pelo usuário.

Código SQL completo

```
-- ========================================
-- SISTEMA DE CONTROLE FINANCEIRO BÁSICO
-- Banco de Dados: MySQL
-- ========================================

-- 1. Criar o banco de dados
CREATE DATABASE controle_financeiro;

-- 2. Selecionar o banco de dados
USE controle_financeiro;

-- 3. Criar a tabela de categorias
CREATE TABLE categorias (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    tipo ENUM('RECEITA', 'DESPESA') NOT NULL
);

-- 4. Criar a tabela de lançamentos financeiros
CREATE TABLE lancamentos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    descricao VARCHAR(150) NOT NULL,
    valor DECIMAL(10, 2) NOT NULL,
    tipo ENUM('RECEITA', 'DESPESA') NOT NULL,
    data_lancamento DATE NOT NULL,
    categoria_id INT NOT NULL,

    CONSTRAINT fk_categoria
        FOREIGN KEY (categoria_id)
        REFERENCES categorias(id)
);

-- 5. Inserir categorias de receita
INSERT INTO categorias (nome, tipo) VALUES
('Salário', 'RECEITA'),
('Freelance', 'RECEITA'),
('Investimentos', 'RECEITA');

-- 6. Inserir categorias de despesa
INSERT INTO categorias (nome, tipo) VALUES
('Alimentação', 'DESPESA'),
```

```
('Transporte', 'DESPESA'),
('Moradia', 'DESPESA'),
('Lazer', 'DESPESA'),
('Saúde', 'DESPESA');

-- 7. Inserir alguns lançamentos financeiros
INSERT INTO lancamentos
(descricao, valor, tipo, data_lancamento, categoria_id)
VALUES
('Salário mensal', 3500.00, 'RECEITA', '2026-11-05', 1),
('Projeto freelance', 800.00, 'RECEITA', '2026-11-08', 2),
('Supermercado', 450.75, 'DESPESA', '2026-11-06', 4),
('Combustível', 250.00, 'DESPESA', '2026-11-07', 5),
('Aluguel', 1200.00, 'DESPESA', '2026-11-10', 6);

-- 8. Consultar todas as categorias
SELECT * FROM categorias;

-- 9. Consultar todos os lançamentos
SELECT * FROM lancamentos;

-- 10. Consultar lançamentos junto com suas categorias
SELECT
    l.id,
    l.descricao,
    l.valor,
    l.tipo,
    l.data_lancamento,
    c.nome AS categoria
FROM lancamentos l
INNER JOIN categorias c
    ON l.categoria_id = c.id;

-- 11. Calcular o total de receitas
SELECT SUM(valor) AS total_receitas
FROM lancamentos
WHERE tipo = 'RECEITA';

-- 12. Calcular o total de despesas
SELECT SUM(valor) AS total_despesas
FROM lancamentos
WHERE tipo = 'DESPESA';

-- 13. Calcular o saldo financeiro
SELECT
    SUM(
        CASE
            WHEN tipo = 'RECEITA' THEN valor
            WHEN tipo = 'DESPESA' THEN -valor
        END
    ) AS saldo
FROM lancamentos;

-- 14. Listar apenas despesas
SELECT *
FROM lancamentos
WHERE tipo = 'DESPESA'
ORDER BY valor DESC;

-- 15. Atualizar um lançamento
UPDATE lancamentos
SET valor = 500.00
WHERE id = 3;

-- 16. Excluir um lançamento
DELETE FROM lancamentos
WHERE id = 5;
```

Explicação passo a passo

1. CREATE DATABASE

O comando **CREATE DATABASE controle_financeiro;** cria um novo banco de dados chamado **controle_financeiro**. O banco é o local onde todas as tabelas do sistema serão organizadas.

2. USE

O comando **USE controle_financeiro;** informa ao MySQL que os próximos comandos devem ser executados dentro desse banco de dados. Sem selecionar o banco, o MySQL não sabe onde criar as tabelas.

3. CREATE TABLE categorias

Cria a tabela responsável pelas categorias financeiras. O campo **id** identifica cada categoria. **INT** representa números inteiros; **AUTO_INCREMENT** gera automaticamente o próximo número; **PRIMARY KEY** define a chave primária. O campo **nome** recebe até 100 caracteres e **NOT NULL** impede valores vazios. O campo **tipo** usa **ENUM**, permitindo apenas RECEITA ou DESPESA.

4. CREATE TABLE lancamentos

Cria a tabela principal do sistema. Cada registro representa uma movimentação financeira. A descrição explica o lançamento, o valor guarda o dinheiro movimentado, o tipo identifica se é receita ou despesa, a data registra quando ocorreu e **categoria_id** relaciona o lançamento com uma categoria.

5. DECIMAL(10,2)

Valores financeiros não devem ser armazenados normalmente em FLOAT, pois podem ocorrer problemas de precisão. **DECIMAL(10,2)** permite até 10 dígitos no total, sendo 2 deles após a vírgula decimal.

6. FOREIGN KEY

A restrição **FOREIGN KEY (categoria_id) REFERENCES categorias(id)** cria o relacionamento entre as tabelas. Isso significa que um lançamento deve apontar para uma categoria existente.

7. INSERT INTO

Os comandos INSERT adicionam dados às tabelas. Primeiro são cadastradas as categorias e, depois, os lançamentos. Os vários valores separados por vírgula permitem inserir diversos registros com um único comando INSERT.

8. SELECT *

O comando **SELECT *** consulta todos os campos de uma tabela. É útil para visualizar rapidamente os dados cadastrados.

9. INNER JOIN

O JOIN combina informações de tabelas relacionadas. Neste caso, ele mostra o lançamento junto com o nome da categoria. A condição **ON l.categoria_id = c.id** informa como os registros devem ser relacionados.

10. AS categoria

A palavra **AS** cria um apelido para uma coluna no resultado da consulta. Assim, **c.nome AS categoria** faz a coluna aparecer com o nome 'categoria'.

11. SUM + WHERE

SUM(valor) soma os valores encontrados. A cláusula **WHERE tipo = 'RECEITA'** filtra apenas as receitas. O mesmo conceito é utilizado para calcular o total das despesas.

12. CASE para calcular o saldo

O **CASE** funciona como uma decisão dentro do SQL. Se o lançamento for RECEITA, o valor é somado positivamente. Se for DESPESA, o valor recebe sinal negativo. Depois, SUM calcula o saldo final.

13. ORDER BY

ORDER BY valor DESC organiza os resultados pelo valor, do maior para o menor. **DESC** significa ordem decrescente.

14. UPDATE

UPDATE altera registros existentes. O comando usado muda o valor do lançamento cujo id é 3. A cláusula **WHERE** é extremamente importante para evitar alterar todos os registros.

15. DELETE

DELETE remove registros. No exemplo, o lançamento de id 5 é excluído. Assim como no UPDATE, utilizar WHERE é essencial: um DELETE sem WHERE pode apagar todos os registros da tabela.

Desafio para os alunos

Depois de executar o script inicial, os alunos devem evoluir o sistema realizando as atividades abaixo:

1. Cadastrar uma nova categoria chamada **Educação**.

2. Cadastrar pelo menos 5 novos lançamentos financeiros.

3. Criar uma consulta que mostre apenas as receitas acima de R$ 1.000,00.

4. Criar uma consulta que mostre despesas por categoria.

5. Calcular o total gasto em uma categoria específica.

6. Listar os lançamentos em ordem de data, do mais recente para o mais antigo.

7. Alterar a descrição de um lançamento utilizando UPDATE.

8. Excluir um lançamento de teste utilizando DELETE.

9. Desafio avançado: criar uma consulta que mostre o total de receitas, despesas e saldo em uma única visão.

Resumo dos principais comandos

Comando	Função
CREATE DATABASE	Cria um banco de dados.
USE	Seleciona o banco que será utilizado.
CREATE TABLE	Cria uma tabela.
INSERT INTO	Adiciona registros.
SELECT	Consulta dados.
WHERE	Filtra registros.
INNER JOIN	Relaciona informações entre tabelas.
SUM	Realiza soma de valores.
ORDER BY	Ordena os resultados.
UPDATE	Atualiza registros existentes.
DELETE	Remove registros.

Projeto sugerido: os alunos podem utilizar este banco como backend de uma futura aplicação em Node.js, JavaScript/TypeScript ou outra tecnologia estudada no curso.